

Task:1

Aim: Write a programme to conduct uninformed and informed search.

What is an Uninformed Search

Uninformed search is one in which the search systems do not use any clues about the suitable area but it depend on the random nature of search. Nevertheless, they begin the exploration of search space (all possible solutions) synchronously. The search operation begins from the initial state and providing all possible next steps arrangement until goal is reached. These are mostly the simplest search strategies, but they may not be suitable for complex paths which involve in irrelevant or even irrelevant components. These algorithms are necessary for solving basic tasks or providing simple processing before passing on the data to more advanced search algorithms that incorporate prioritized information.

Examples: Depth First Search and Breadth-First Search

Program (Uninformed Search): DFS Program for Uninformed Search

```
def Check_Visited(visited_nodes, current_node):
    if current_node not in visited_nodes:
        visited_nodes.append(current_node)
        print(current_node , end=' -> ')
def Search_DFS(graph, start, destination):
    visited_nodes = []
    stack = [start]
    while stack:
        current_node = stack.pop()
        Check_Visited(visited_nodes, current_node)
        if current_node == destination:
            return
        for adjacent_node in graph[current_node]:
            if adjacent_node not in visited_nodes:
                stack.append(adjacent_node)
my_graph = {
    'A': set(['F', 'C']),
    'B': set(['D', 'E']),
    'C': set(['B']),
    'D': set(['H']),
    'E': set([]),
    'F': set(['G']),
    'G': set([]),
}
print("DFS for graph: \n", my_graph)
print("Path to be followed is")
```

```
Search_DFS(my_graph, 'A', 'H')
```

Output:

DFS for graph:

```
{'A': {'C', 'F'}, 'B': {'E', 'D'}, 'C': {'B'}, 'D': {'H'}, 'E': set(), 'F': {'G'}, 'G': set()}
```

Path to be followed is

```
A -> F -> G -> C -> B -> D -> H ->
```

Program (Uninformed Search):

BFS Program for Uninformed Search

```
def Check_Visited(visited_nodes, current_node):
```

```
    if current_node not in visited_nodes:
```

```
        visited_nodes.append(current_node)
```

```
        print(current_node, end=' -> ')
```

```
def Search_BFS(graph, start, destination):
```

```
    visited_nodes = []
```

```
    queue = [start]
```

```
    visited_nodes.append(start)
```

```
    print(start, end=' -> ')
```

```
    while queue:
```

```
        current_node = queue.pop(0)
```

```
        if current_node == destination:
```

```
            return
```

```
        for adjacent_node in graph[current_node]:
```

```
            if adjacent_node not in visited_nodes:
```

```
                visited_nodes.append(adjacent_node)
```

```
                print(adjacent_node, end=' -> ')
```

```
                queue.append(adjacent_node)
```

```
my_graph = {
```

```
    'A': set(['B', 'C']),
```

```
    'B': set(['D', 'E']),
```

```
    'C': set(['F']),
```

```
    'D': set(['H']),
```

```
    'E': set([]),
```

```
    'F': set(['G']),
```

```
    'G': set([]),
```

```
}
```

```
print("BFS for graph: \n", my_graph)
```

```
print("Path to be followed is")
```

```
Search_BFS(my_graph, 'A', 'H')
```

Output

BFS for graph:

```
{'A': {'D', 'C'}, 'B': {'B', 'E'}, 'C': {'F'}, 'D': {'H'}, 'E': set(), 'F': {'G'}, 'G': set()}
```

Path to be followed is

```
A -> D -> C -> H -> F ->
```

What is an Informed Search

Informed search algorithms, also known as heuristic search algorithms, are an essential component of Artificial Intelligence (AI). These algorithms use domain-specific knowledge to improve the efficiency of the search process, leading to faster and more optimal solutions compared to uninformed search methods. By incorporating heuristics, informed search algorithms can make educated guesses about which paths to explore in a search space, ultimately reducing the time and computational resources required to find a solution.

Example: Greedy Search and Graph Search, A* Search.

Program (informed Search):

A* Program for Informed Search

```
def FindOptimalSequence(closed):
    trace_node = 'G'
    optimal_sequence = ['G']
    for i in range(len(closed) - 2, -1, -1):
        check_node = closed[i][0]
        if trace_node in [children[0] for children in tree2[check_node]]:
            children_costs = [temp[1] for temp in tree2[check_node]]
            children_nodes = [temp[0] for temp in tree2[check_node]]
            if cost[check_node] + children_costs[children_nodes.index(trace_node)] ==
cost[trace_node]:
                optimal_sequence.append(check_node)
                trace_node = check_node
    optimal_sequence.reverse()
    return closed, optimal_sequence

def Search_A_Star():
    global tree, heuristic
    closed = []
    opened = [['S', 8]] # Initialize with the start node 'S' and its heuristic value
    # Finding the visited nodes
    while True:
        fn = [i[1] for i in opened]
        chosen_index = fn.index(min(fn))
        node = opened[chosen_index][0]
        closed.append(opened[chosen_index])
        del opened[chosen_index]
        if closed[-1][0] == 'G': # Check if the goal node 'G' is reached
            break
        for item in tree2[node]:
```

```

        if item[0] in [closed_item[0] for closed_item in closed]:
            continue
        cost.update({item[0]: cost[node] + item[1]})
        fn_node = cost[node] + heuristic[item[0]] + item[1]
        temp = [item[0], fn_node]
        opened.append(temp)
    closed, optimal = FindOptimalSequence(closed)
    return closed, optimal

# Define the tree, heuristic, and cost
tree2 = {
    'S': [['A', 1], ['B', 2]],
    'A': [['S', 1]],
    'B': [['S', 2], ['C', 3], ['D', 4]],
    'C': [['B', 2], ['E', 5], ['F', 6]],
    'D': [['B', 4], ['G', 7]],
    'E': [['C', 5]],
    'F': [['C', 6]],
}

heuristic = {
    'S': 0,
    'A': 5000,
    'B': 2,
    'C': 3,
    'D': 4,
    'E': 5000,
    'F': 5000,
    'G': 0
}

cost = {
    'S': 0
}

# Execute the A* Search and find the optimal sequence
visited_nodes, optimal_nodes = Search_A_Star()
print('The Sequence of the Visited Nodes is:\n ' + str(visited_nodes))
print('The Optimal Road Sequence is:\n ' + str(optimal_nodes))

```

Output:

```

The Sequence of the Visited Nodes is:
[['S', 8], ['B', 4], ['C', 8], ['D', 10], ['G', 13]]
The Optimal Road Sequence is:
['S', 'B', 'D', 'G']

```

Task:2

Aim: Write a programme to conduct game search.

Game Search in AI

Game search refers to the process of finding the best possible move or sequence of moves in a game environment. In AI, this is achieved by exploring the game's possible states (also known as the game tree) and evaluating them using certain strategies or algorithms.

The core idea of game search is to simulate all possible future moves and their consequences, and then choose the move that leads to the most favorable outcome, assuming both players are playing optimally.

Key Concepts of Game Search:

Game State: A game state represents a snapshot of the game at any point in time. It includes information like the positions of the pieces, whose turn it is, and other relevant game-specific data. In Tic-Tac-Toe, for example, a game state would include the configuration of the 3x3 grid, indicating whether the cell is occupied by 'X', 'O', or is empty.

Game Tree: A game tree is a tree-like structure where each node represents a possible game state. The root node represents the current game state, and each branch represents a possible move. For example, in Tic-Tac-Toe, starting from the initial empty grid, the game tree branches out to all possible moves of the human player, and for each of those moves, it further branches out to all possible moves of the AI, and so on.

Players:

Maximizing player: The player who is trying to maximize the outcome of the game, often representing the AI. In many games, this is the AI player or the computer, which tries to make the best move.

Minimizing player: The opponent who is trying to minimize the maximizing player's chances of winning. In a game like Tic-Tac-Toe, this is the human player ('X') trying to block the AI ('O').

Terminal States: These are the end states of the game. A terminal state indicates that the game is over, either because there's a winner or because the game has ended in a tie.

In Tic-Tac-Toe, terminal states are reached when one player wins (three consecutive marks) or the board is full without a winner (a tie).

Game Search Algorithms

The purpose of game search algorithms is to explore all the possible states of the game and evaluate them to choose the optimal move. Below are the main algorithms used for game search:

1. Minimax Algorithm: The Minimax algorithm is one of the fundamental search algorithms used in two-player games. It's a recursive algorithm that explores all possible moves and selects the optimal move for the player. The algorithm assumes that both players are playing optimally, meaning each player is trying to maximize their score or minimize the opponent's score.

2. Alpha-Beta Pruning: Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It helps to prune (cut off) branches of the game tree that are unlikely to influence the final decision, thereby improving the performance and reducing the number of nodes evaluated.

3. Depth-First Search (DFS) and Breadth-First Search (BFS)

While Minimax and Alpha-Beta Pruning are more focused on game-theoretic evaluations, DFS and BFS are more general search algorithms. However, they can be adapted to explore a game tree:

Depth-First Search (DFS): Explores as deeply as possible along one branch before backtracking. This can be useful in games where we want to explore one long chain of moves before considering alternatives.

Breadth-First Search (BFS): Explores all possible moves at the current depth level before moving to the next. This ensures that all possible outcomes are considered level by level.

Conclusion:

Game search is a critical concept in AI for games, as it allows AI agents to make intelligent decisions by exploring the possible game states and choosing optimal moves. The most common game search techniques include:

Minimax: A strategy for two-player zero-sum games, where one player aims to maximize their score and the other minimizes it.

Alpha-Beta Pruning: An optimization to Minimax that cuts off branches of the search tree that do not affect the final outcome.

Depth-First Search and Breadth-First Search: More general search algorithms that explore game states in a systematic way.

These techniques can be applied to games ranging from simple ones like Tic-Tac-Toe to complex ones like Chess and Go, where AI strategies heavily rely on evaluating and exploring the game tree.

PROGRAM

```
import math
class TicTacToeAI:
    def __init__(self):
        # Initialize the board as a 3x3 grid, filled with empty spaces
        self.board = [['.' for _ in range(3)] for _ in range(3)]
```

```

self.current_player = 'X' # 'X' is the player, 'O' is the AI
self.opponent = 'O'

def print_board(self):
    for row in self.board:
        print(" | ".join(row))
        print("-" * 5)
def is_valid_move(self, x, y):
    # Check if the cell is within the bounds of the grid and empty
    return 0 <= x < 3 and 0 <= y < 3 and self.board[x][y] == '.'
def is_winner(self, player):
    # Check all rows, columns, and diagonals for a winning condition
    for i in range(3):
        if all(self.board[i][j] == player for j in range(3)): # Check row
            return True
        if all(self.board[j][i] == player for j in range(3)): # Check column
            return True
    if all(self.board[i][i] == player for i in range(3)): # Check main diagonal
        return True
    if all(self.board[i][2-i] == player for i in range(3)): # Check anti-diagonal
        return True
    return False
def is_full(self):
    # Check if the board is full
    return all(self.board[i][j] != '.' for i in range(3) for j in range(3))
def minimax(self, depth, is_maximizing, alpha, beta):
    # If the game is over, return the score based on the result
    if self.is_winner(self.opponent):
        return 1 # AI wins
    elif self.is_winner(self.current_player):
        return -1 # Player wins
    elif self.is_full():
        return 0 # Tie
    # Maximize or minimize based on the current player
    if is_maximizing:
        max_eval = -math.inf
        for i in range(3):
            for j in range(3):
                if self.is_valid_move(i, j):
                    self.board[i][j] = self.opponent
                    eval = self.minimax(depth + 1, False, alpha, beta)
                    self.board[i][j] = '.' # Undo move
                    max_eval = max(max_eval, eval)
                    alpha = max(alpha, eval)
                    if beta <= alpha:
                        break # Alpha-Beta pruning

```

```

        return max_eval
    else:
        min_eval = math.inf
        for i in range(3):
            for j in range(3):
                if self.is_valid_move(i, j):
                    self.board[i][j] = self.current_player
                    eval = self.minimax(depth + 1, True, alpha, beta)
                    self.board[i][j] = '.' # Undo move
                    min_eval = min(min_eval, eval)
                    beta = min(beta, eval)
                    if beta <= alpha:
                        break # Alpha-Beta pruning
            return min_eval
def best_move(self):
    best_value = -math.inf
    move = (-1, -1)
    # Try all possible moves and evaluate them using Minimax
    for i in range(3):
        for j in range(3):
            if self.is_valid_move(i, j):
                self.board[i][j] = self.opponent # AI's move
                move_value = self.minimax(0, False, -math.inf, math.inf)
                self.board[i][j] = '.' # Undo the move

                # If this move is better than the previous ones, update the best move
                if move_value > best_value:
                    best_value = move_value
                    move = (i, j)
    return move
def play_game(self):
    # Main game loop
    while not self.is_winner(self.opponent) and not self.is_winner(self.current_player) and not self.is_full():
        self.print_board()
        if self.current_player == 'X':
            x, y = map(int, input("Enter your move (row col): ").split())
            if self.is_valid_move(x, y):
                self.board[x][y] = 'X'
                self.current_player = 'O'
        else:
            print("AI is making its move...")
            x, y = self.best_move() # Get AI's best move
            self.board[x][y] = 'O'
            self.current_player = 'X'
    # Print final board and determine winner

```



```

    self.print_board()
    if self.is_winner(self.opponent):
        print("AI wins!")
    elif self.is_winner(self.current_player):
        print("Player wins!")
    else:
        print("It's a tie!")
# Initialize and play the game
if __name__ == "__main__":
    game = TicTacToeAI()
    game.play_game()
0 1

```

OUTPUT

```

. | . | .
-----
. | . | .
-----
. | . | .
-----
Enter your move (row col): 0 1
. | X | .
-----
. | . | .
-----
. | . | .
-----
AI is making its move...
O | X | .
-----
. | . | .
-----
. | . | .
-----
Enter your move (row col): 1 1
O | X | .
-----
. | X | .
-----
. | . | .
-----
AI is making its move...
O | X | O
-----
. | X | .
-----
. | . | .
-----
Enter your move (row col): 2 1
O | X | O
-----
. | X | .
-----
. | X | .
-----
AI is making its move...
O | X | O
-----
O | X | .
-----
. | X | .
-----
Player wins!

```

Task:3

Aim: Write a programme to construct a Bayesian network from given data.

Bayesian Networks, a concept in Artificial Intelligence (AI), are graphical models that represent the probabilistic relationships among a set of variables. They are crucial for understanding uncertainty & making informed decisions in complex systems. Their importance lies in their ability to model conditional dependencies, which is fundamental in many AI applications.

PROGRAM

```
import pandas as pd
data=pd.read_csv("heartdisease.csv")
heart_disease=pd.DataFrame(data)
print(heart_disease)
```

	age	Gender	Family	diet	Lifestyle	cholesterol	heartdisease
0	0	0	1	1	3	0	1
1	0	1	1	1	3	0	1
2	1	0	0	0	2	1	1
3	4	0	1	1	3	2	0
4	3	1	1	0	0	2	0
5	2	0	1	1	1	0	1
6	4	0	1	0	2	0	1
7	0	0	1	1	3	0	1
8	3	1	1	0	0	2	0
9	1	1	0	0	0	2	1
10	4	1	0	1	2	0	1
11	4	0	1	1	3	2	0
12	2	1	0	0	0	0	0
13	2	0	1	1	1	0	1
14	3	1	1	0	0	1	0
15	0	0	1	0	0	2	1
16	1	1	0	1	2	1	1
17	3	1	1	1	0	1	0
18	4	0	1	1	3	2	0

```
from pgmpy.models import BayesianModel
model=BayesianModel([
('age','Lifestyle'),
('Gender','Lifestyle'),
('Family','heartdisease'),
('diet','cholesterol'),
('Lifestyle','diet'),
('cholesterol','heartdisease'),
('diet','cholesterol')
])
from pgmpy.estimators import MaximumLikelihoodEstimator
model.fit(heart_disease, estimator=MaximumLikelihoodEstimator)
from pgmpy.inference import VariableElimination
HeartDisease_infer = VariableElimination(model)
print('For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4 }')
print('For Gender Enter { Male:0, Female:1 }')
print('For Family History Enter { yes:1, No:0 }')
print('For diet Enter { High:0, Medium:1 }')
print('For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }')
print('For cholesterol Enter { High:0, BorderLine:1, Normal:2 }')
```

```

q = HeartDisease_infer.query(variables=['heartdisease'], evidence={ 'age':int(input('Enter age :'))
'Gender':int(input('Enter Gender :')),
'Family':int(input('Enter Family history :')),
'diet':int(input('Enter diet :')),
'Lifestyle':int(input('Enter Lifestyle :')),
'cholesterol':int(input('Enter cholesterol :'))
})

```

```

print(q['heartdisease'])

```

```

For age Enter { SuperSeniorCitizen:0, SeniorCitizen:1, MiddleAged:2, Youth:3, Teen:4 }
For Gender Enter { Male:0, Female:1 }
For Family History Enter { yes:1, No:0 }
For diet Enter { High:0, Medium:1 }
For lifeStyle Enter { Athlete:0, Active:1, Moderate:2, Sedentary:3 }
For cholesterol Enter { High:0, BorderLine:1, Normal:2 }
Enter age :1
Enter Gender :1
Enter Family history :0
Enter diet :1
Enter Lifestyle :0
Enter cholesterol :1
+-----+
| heartdisease | phi(heartdisease) |
+-----+
| heartdisease_0 | 0.0000 |
+-----+
| heartdisease_1 | 1.0000 |
+-----+

```

Task:4

Aim: Write a programme to infer from the Bayesian network.

A Bayesian network (also spelt Bayes network, Bayes net, belief network, or judgment network) is a probabilistic graphical model that depicts a set of variables and their conditional dependencies using a directed acyclic graph (DAG).

Bayesian networks are perfect for taking an observed event and forecasting the likelihood that any of numerous known causes played a role. A Bayesian network, for example, could reflect the probability correlations between diseases and symptoms. Given a set of symptoms, the network may be used to calculate the likelihood of the presence of certain diseases.

Inferencing with Bayesian Network in Python

PROGRAM

Variables:

- B: Burglary
- E: Earthquake
- A: Alarm (depends on B and E)
- J: JohnCalls (depends on A)
- M: MaryCalls (depends on A)

Dependencies:

- $P(B)$, $P(E)$ — Prior probabilities
- $P(A | B, E)$ — Alarm given Burglary and Earthquake
- $P(J | A)$, $P(M | A)$ — Calls given Alarm

Goal

Let's compute a probability like:

$P(\text{Burglary} | \text{JohnCalls} = \text{True}, \text{MaryCalls} = \text{True})$

This uses Bayesian inference.

```
from collections import defaultdict
```

```
# Define the Bayesian Network structure and probabilities
```

```
bn = {  
    'B': {'parents': [], 'prob': lambda vals: 0.001},  
    'E': {'parents': [], 'prob': lambda vals: 0.002},  
    'A': {  
        'parents': ['B', 'E'],  
        'prob': lambda vals: {  
            (True, True): 0.95,
```

```

        (True, False): 0.94,
        (False, True): 0.29,
        (False, False): 0.001
    }[(vals['B'], vals['E'])]
},
'J': {
    'parents': ['A'],
    'prob': lambda vals: {True: 0.90, False: 0.05}[vals['A']]
},
'M': {
    'parents': ['A'],
    'prob': lambda vals: {True: 0.70, False: 0.01}[vals['A']]
}
}

```

```

def enumerate_all(variables, evidence):
    if not variables:
        return 1.0
    first, rest = variables[0], variables[1:]
    if first in evidence:
        prob = bn[first]['prob'](evidence)
        return prob * enumerate_all(rest, evidence)
    else:
        total = 0
        for val in [True, False]:
            extended_evidence = dict(evidence)
            extended_evidence[first] = val
            prob = bn[first]['prob'](extended_evidence)
            total += prob * enumerate_all(rest, extended_evidence)
        return total

def query(var, evidence):
    probs = {}
    for val in [True, False]:
        extended_evidence = dict(evidence)

```

```
    extended_evidence[var] = val
    probs[val] = enumerate_all(list(bn.keys()), extended_evidence)
# Normalize
total = sum(probs.values())
for k in probs:
    probs[k] /= total
return probs

# Example query: P(B | J=True, M=True)
result = query('B', {'J': True, 'M': True})
print("P(Burglary | JohnCalls=True, MaryCalls=True):")
print(result)
```

OUTPUT

```
P(Burglary | JohnCalls=True, MaryCalls=True):
{True: 0.8665749656121045, False: 0.13342503438789546}
```

Task: 5

Aim: Write a programme to run value and policy iteration in a grid world.

Introduction:

In a grid world, the environment is represented as a grid of cells, where each cell can be either an obstacle or a reward. The agent can move through the grid by taking actions such as moving up, down, left, or right. The goal is to find the optimal policy, which is a mapping from states to actions that maximizes the expected cumulative reward over time.

Value iteration is an iterative algorithm that computes the optimal value function, which is the expected cumulative reward from each state under the optimal policy. The algorithm starts with an initial estimate of the value function and then iteratively updates the value function using the Bellman optimality equation until it converges to the true optimal value function. The Bellman optimality equation is:

$$V(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')]$$

where $V(s)$ is the value of state s , $R(s,a)$ is the immediate reward of taking action a in state s , Factor.

Policy iteration is another iterative algorithm that alternates between improving the policy and evaluating the policy. The algorithm starts with an initial policy and then iteratively updates the policy by greedily selecting the action that maximizes the expected cumulative reward under the current value function, and then evaluates the policy by computing the value function under the policy using the Bellman equation. The algorithm terminates when the policy no longer changes.

PROGRAM

```
import gym
import numpy as np

# Initialize the environment
env = gym.make('FrozenLake-v0')

# Policy Evaluation
def policy_evaluation(policy, environment, discount_factor=1.0, theta=1e-9, max_iterations=1e9):
    evaluation_iterations = 1
    V = np.zeros(environment.nS)
    for i in range(int(max_iterations)):
        delta = 0
        for state in range(environment.nS):
            v = 0
            for action, action_probability in enumerate(policy[state]):
                for state_probability, next_state, reward, terminated in environment.P[state][action]:
                    v += action_probability * state_probability * (reward + discount_factor *
V[next_state])
            delta = max(delta, np.abs(V[state] - v))
        V[state] = v
```

```

evaluation_iterations += 1
if delta < theta:
    print(f'Policy evaluated in {evaluation_iterations} iterations.')
    return V

```

One-Step Lookahead for Policy Improvement

```

def one_step_lookahead(environment, state, V, discount_factor):
    action_values = np.zeros(environment.nA)
    for action in range(environment.nA):
        for probability, next_state, reward, terminated in environment.P[state][action]:
            action_values[action] += probability * (reward + discount_factor * V[next_state])
    return action_values

```

Policy Iteration

```

def policy_iteration(environment, discount_factor=1.0, max_iterations=1e9):
    policy = np.ones((environment.nS, environment.nA)) / environment.nA
    evaluated_policies = 1
    for i in range(int(max_iterations)):
        stable_policy = True
        V = policy_evaluation(policy, environment, discount_factor=discount_factor)
        for state in range(environment.nS):
            current_action = np.argmax(policy[state])
            action_value = one_step_lookahead(environment, state, V, discount_factor)
            best_action = np.argmax(action_value)
            if current_action != best_action:
                stable_policy = False
                policy[state] = np.eye(environment.nA)[best_action]
        evaluated_policies += 1
    if stable_policy:
        print(f'Evaluated {evaluated_policies} policies.')
        return policy, V

```

Value Iteration

```

def value_iteration(environment, discount_factor=1.0, theta=1e-9, max_iterations=1e9):
    V = np.zeros(environment.nS)
    for i in range(int(max_iterations)):
        delta = 0
        for state in range(environment.nS):
            action_value = one_step_lookahead(environment, state, V, discount_factor)
            best_action_value = np.max(action_value)
            delta = max(delta, np.abs(V[state] - best_action_value))
            V[state] = best_action_value
        if delta < theta:
            print(f'Value iteration converged after {i + 1} iterations.')
            break
    policy = np.zeros((environment.nS, environment.nA))
    for state in range(environment.nS):
        action_value = one_step_lookahead(environment, state, V, discount_factor)
        best_action = np.argmax(action_value)
        policy[state, best_action] = 1.0
    return policy, V

```



```

# Simulate Episodes
def play_episodes(environment, n_episodes, policy):
    wins = 0
    total_reward = 0
    for episode in range(n_episodes):
        terminated = False
        state = environment.reset()
        while not terminated:
            action = np.argmax(policy[state])
            next_state, reward, terminated, info = environment.step(action)
            total_reward += reward
            state = next_state
            if terminated and reward == 1.0:
                wins += 1
        average_reward = total_reward / n_episodes
    return wins, total_reward, average_reward

# Main Code
n_episodes = 1000
solvers = [('Policy Iteration', policy_iteration),
            ('Value Iteration', value_iteration)]
for iteration_name, iteration_func in solvers:
    environment = gym.make('FrozenLake-v0')
    policy, V = iteration_func(environment.env)
    wins, total_reward, average_reward = play_episodes(environment, n_episodes, policy)
    print(f'{iteration_name} :: number of wins over {n_episodes} episodes = {wins}')
    print(f'{iteration_name} :: average reward over {n_episodes} episodes = {average_reward}\n\n')

```

OUTPUT

```

Policy evaluated in 66 iterations.
Evaluated 2 policies.
Policy Iteration :: number of wins over 1000 episodes = 713
Policy Iteration :: average reward over 1000 episodes = 0.713

```

```

Value-iteration converged at iteration#523.
Value Iteration :: number of wins over 1000 episodes = 734
Value Iteration :: average reward over 1000 episodes = 0.734

```

Task: 6

Aim: Write a programme to do reinforcement learning in a grid world.

Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards. RL allows machines to learn by interacting with an environment and receiving feedback based on their actions. This feedback comes in the form of rewards or penalties.

Reinforcement Learning revolves around the idea that an agent (the learner or decision-maker) interacts with an environment to achieve a goal. The agent performs actions and receives feedback to optimize its decision-making over time.

- **Agent:** The decision-maker that performs actions.
- **Environment:** The world or system in which the agent operates.
- **State:** The situation or condition the agent is currently in.
- **Action:** The possible moves or decisions the agent can make.
- **Reward:** The feedback or result from the environment based on the agent's action.

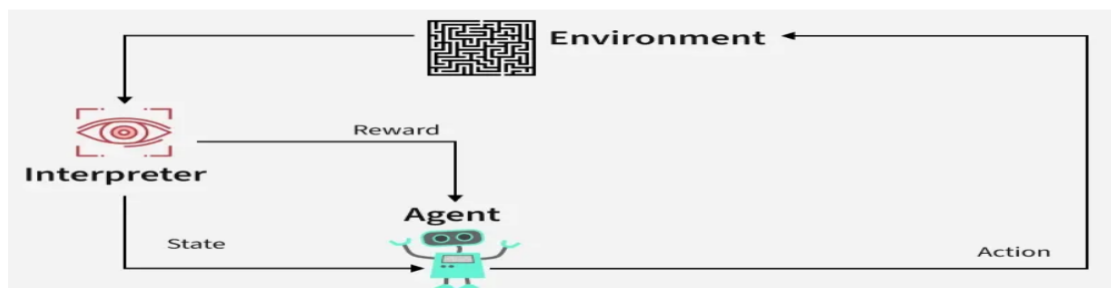


Figure 1

How Reinforcement Learning Works?

The RL process involves an agent performing actions in an environment, receiving rewards or penalties based on those actions, and adjusting its behavior accordingly. This loop helps the agent improve its decision-making over time to maximize the **cumulative reward**.

Here's a breakdown of RL components:

- **Policy:** A strategy that the agent uses to determine the next action based on the current state.
- **Reward Function:** A function that provides feedback on the actions taken, guiding the agent towards its goal.
- **Value Function:** Estimates the future cumulative rewards the agent will receive from a given state.
- **Model of the Environment:** A representation of the environment that predicts future states and rewards, aiding in planning.

PROGRAM

```
import random
from typing import List
class SampleEnvironment:
    def __init__(self):
        self.steps_left = 10
    def get_observations(self) -> List[float]:
        self.my_observation = random.randint(1, 100)
        observed_value = float(self.my_observation)
        if self.my_observation <= 33:
            return [observed_value, 0.0, 0.0]
        elif 33 < self.my_observation <= 66:
            return [0.0, observed_value, 0.0]
        else:
            return [0.0, 0.0, observed_value]
    def get_actions(self) -> List[int]:
        if self.my_observation <= 33:
            return [1, 0, 0]
        elif 33 < self.my_observation <= 66:
            return [0, 1, 0]
        else:
            return [0, 0, 1]
    def is_done(self) -> bool:
        return self.steps_left == 0
    def action(self, action: int) -> float: # Define the action method
        if self.is_done():
            raise Exception("Game is over")
        self.steps_left -= 1
        return random.random() # Return a random reward
class Agent:
    def __init__(self) -> None:
        self.total_reward = 0.0
    def step(self, env: SampleEnvironment):
        current_obs = env.get_observations()
        print("Current Observation: {}".format(current_obs))
        actions = env.get_actions()
        if actions == [0, 1, 0]:
            print("Current Action: {} : NO TURN".format(actions))
        elif actions == [1, 0, 0]:
            print("Current Action: {} : LEFT TURN".format(actions))
        else:
            print("Current Action: {} : RIGHT TURN".format(actions))
        reward = env.action(random.choice(actions)) # Use the newly added method
        self.total_reward += reward
        print("Reward Collected and State Changed...")
```

```

    print()
if __name__ == "__main__":
    env = SampleEnvironment()
    agent = Agent()
    i = 1
    while not env.is_done():
        print("STEP: ", i)
        i += 1
        agent.step(env)
    print("TOTAL REWARD AT THE END OF GAME IS: %.4f" % agent.total_reward)

```

OUTPUT

```

STEP:  1
Current Observation: [0.0, 0.0, 68.0]
Current Action: [0, 0, 1] : RIGHT TURN
Reward Collected and State Changed...

STEP:  2
Current Observation: [13.0, 0.0, 0.0]
Current Action: [1, 0, 0] : LEFT TURN
Reward Collected and State Changed...

STEP:  3
Current Observation: [0.0, 0.0, 99.0]
Current Action: [0, 0, 1] : RIGHT TURN
Reward Collected and State Changed...

STEP:  4
Current Observation: [0.0, 0.0, 95.0]
Current Action: [0, 0, 1] : RIGHT TURN
Reward Collected and State Changed...

STEP:  5
Current Observation: [0.0, 38.0, 0.0]
Current Action: [0, 1, 0] : NO TURN
Reward Collected and State Changed...

STEP:  6
Current Observation: [33.0, 0.0, 0.0]
Current Action: [1, 0, 0] : LEFT TURN
Reward Collected and State Changed...

STEP:  7
Current Observation: [0.0, 59.0, 0.0]
Current Action: [0, 1, 0] : NO TURN
Reward Collected and State Changed...

STEP:  8
Current Observation: [0.0, 58.0, 0.0]
Current Action: [0, 1, 0] : NO TURN
Reward Collected and State Changed...

STEP:  9
Current Observation: [0.0, 41.0, 0.0]
Current Action: [0, 1, 0] : NO TURN
Reward Collected and State Changed...

STEP:  10
Current Observation: [0.0, 0.0, 89.0]
Current Action: [0, 0, 1] : RIGHT TURN
Reward Collected and State Changed...

TOTAL REWARD AT THE END OF GAME IS: 3.7485

```
